S.G.Ganesh

Bug Hunt

Every programmer knows that debugging is akin to a detective's work. In this column, we'll cover this 'bug hunt' process with an example of how the Intel Pentium processor bug was discovered.

Software can fail in unexpected ways and in the least anticipated situations. In small programs, it is easy to debug, but in large software (often with more than a million lines of code), it is really difficult to debug. Bug hunts are usually enjoyable, because they are challenging. But at times the job can get frustrating, especially when debugging takes many weeks! When the bug is discovered, it is an "Aha!" moment—joy and relief to see the mystery unravelled! Let's look at one instance of a hunt, where a bug was discovered in the hardware.

What was the bug?

Thomas R Nicely, a mathematician, found a flaw in the floating-point division (FDIV) instruction (in the Pentium processor's Floating Point Unit) in 1994. The problem was with five missing entries in the lookup table while implementing the radix-4 SRT algorithm. The bug got exposed only in rare cases. For example, the expression *(8246337024 41.0)*(1/824633702441.0)*, which should equal 1, would get the value *0.999999996274709702* with the Pentium division bug. For typical or normal uses of the computer, one would probably never encounter this bug; however, for scientific computing (like numerical analysis), there were chances of facing this bug. In general, there was a "very small probability of making a very big error" with this bug. This 'Pentium bug' cost Intel hundreds of millions in replacing the chips. We'll look at how Nicely went about his 'bug hunt' before confirming that the bug was in the hardware.

The steps in the hunt

Nicely was working on computational number theory (on prime numbers). He used a number of systems to do calculations, and then, he also added a Pentium machine. In June 1994, he found that the computed values of PI (for a large number of digits) were different from the published value. Nicely first thought that it might be a logic bug or a problem with reduced precision. He also found that the Borland compiler was giving wrong results when some compiler optimisations were enabled. Having disabled the optimisations, and after using *long double* (instead of *double*), he found some new problems. The results for some floating point calculations were different between the Pentium and other hardware. Through trial and error, and doing binary searches to locate the problematic values, he isolated the problem to two prime numbers: 824633702441 and 824633702443. He disabled the optimisations of the Borland compiler, but the error still reproduced. Then he tried disabling the FPU—but made some

Buggy Pentium processor

mistakes, so the FPU did not get disabled. Hence, he thought that the bug was in the PCI bus.

Finally, he purchased a Pentium machine from another manufacturer, which had a different motherboard: the bug still reproduced! When he used Power Basic instead of C, the bug was still there. Then he disabled the FPU unit, and the error disappeared. Finally, he tested the code on yet another Pentium machine, from a different manufacturer, and found the bug occurred on it. With this, Nicely was sure that the bug was in the Pentium FPU!

Lessons for us

We can learn many things from this 'bug hunt': the need for a methodical approach in hunting down bugs; trying to 'isolate' the bug one step at a time; clearly knowing how to reproduce the bug; having the tenacity to keep hunting, and never giving up; never assuming that the bug is in the application we have developed (it might be hidden beneath it)...and so on. END

Resources

- T R Nicely's website: http://www.trnicely.net/

By: S.G. Ganesh

The author works for Siemens (Corporate Technology). He is the author of the best-seller, 'Cracking the C, C++ and Java Interview' published by Tata McGraw-Hill. You can reach him *at sgganesh@gmail.com*.